



Triton Heal: generación de kernels Triton con SLM locales y verificación pre-GPU como filtro de seguridad

Cristóbal Camarena Hernández Josué Galindo Gutiérrez Gandhi Emmanuel Valdez Huerta Diego Alberto Estrada López
Tecnológico de Monterrey — TC3002B (ACA-2026) — Equipo 11

Resumen

Los modelos de lenguaje pequeños (SLM) producen kernels en Triton DSL con errores que solo se descubren al compilar en GPU, un recurso caro. **Triton Heal** propone un pipeline que *genera y luego verifica antes de tocar la GPU*: un SLM local (Llama-3.1-8B) escribe el kernel y, si no pasa las validaciones, un modelo de código (DeepSeek-R1-Distill) lo repara. Evaluamos seis configuraciones sobre **10 tareas de generación** (50 inferencias) y un banco de **70 kernels** de verificación (420 evaluaciones) en un **Mac M4** con Ollama, con diseño *within-subjects* y pruebas pareadas (McNemar / Wilcoxon / Friedman, Holm). DeepSeek-Coder-V2 logró el mejor balance (**100% compilación, MSR 87.1%, FNR 12.9%**); el dual alcanzó **MSR 80.6% / F1 0.86**, mejorando la detección de Llama solo sin superar al mejor frontera. Reportamos con honestidad estadística: la mayoría de contrastes *no* rechaza H_0 tras Holm.

1. Planteamiento del problema

- **Dolor:** compilar y ejecutar kernels Triton en GPU es lento y costoso; un SLM puede generar muchos kernels inválidos que desperdician ese tiempo de cómputo.
- **Línea base:** usar un único modelo grande de frontera por API es caro y depende de conectividad; un SLM local solo es más barato pero falla más y no avisa cuándo se equivoca.
- **Pregunta de investigación:** ¿un esquema *dual* (SLM local + reparación con modelo de código) más un filtro de verificación previo a la GPU mejora la tasa de compilación frente a GPT-4o, y reduce los fallos no detectados (FNR) frente a usar un solo SLM?

2. Antecedentes

- **Triton DSL:** lenguaje de GPU embebido en Python; permite escribir kernels con `@triton.jit` y operaciones `tl.load/tl.store`, más tratable que CUDA crudo.
- **SLM vs. frontera:** los modelos pequeños (8B–14B) corren en local (Ollama, Apple Silicon) pero tienen menor tasa de acierto que modelos de frontera; el reto es cerrar esa brecha sin pagar API.
- **Verificación pre-compilación:** en vez de compilar a ciegas, un *checker* valida sintaxis (AST de Python), reglas léxicas y estructura Triton, y clasifica el kernel *safe/unsafe* (métricas MSR, FNR, F1) antes de gastar la GPU.

3. Nuestro diferenciador

En una línea: mientras otros enfoques apuestan a un solo modelo grande, nosotros combinamos un *SLM local que genera* con un *modelo de código que repara* y un *filtro de verificación pre-GPU*, evaluado con un diseño experimental pre-registrado.

- **Método dual (propio).** Llama-3.1-8B genera; si el checker rechaza, DeepSeek-R1-Distill lo repara (hasta 2 reintentos) y un veto final decide *safe/unsafe*.
- **Rigor experimental.** 6 configuraciones, $T = 0$, semilla 42, diseño *within-subjects*, hipótesis pre-registradas (1.1–1.10) y corrección Holm-Bonferroni sobre la familia de contrastes.
- **Honestidad de resultados.** Declaramos límites: M4 sin CUDA, compilación *sintáctica* (no ejecución GPU), y baselines Claude/GPT.

4. Arquitectura del pipeline

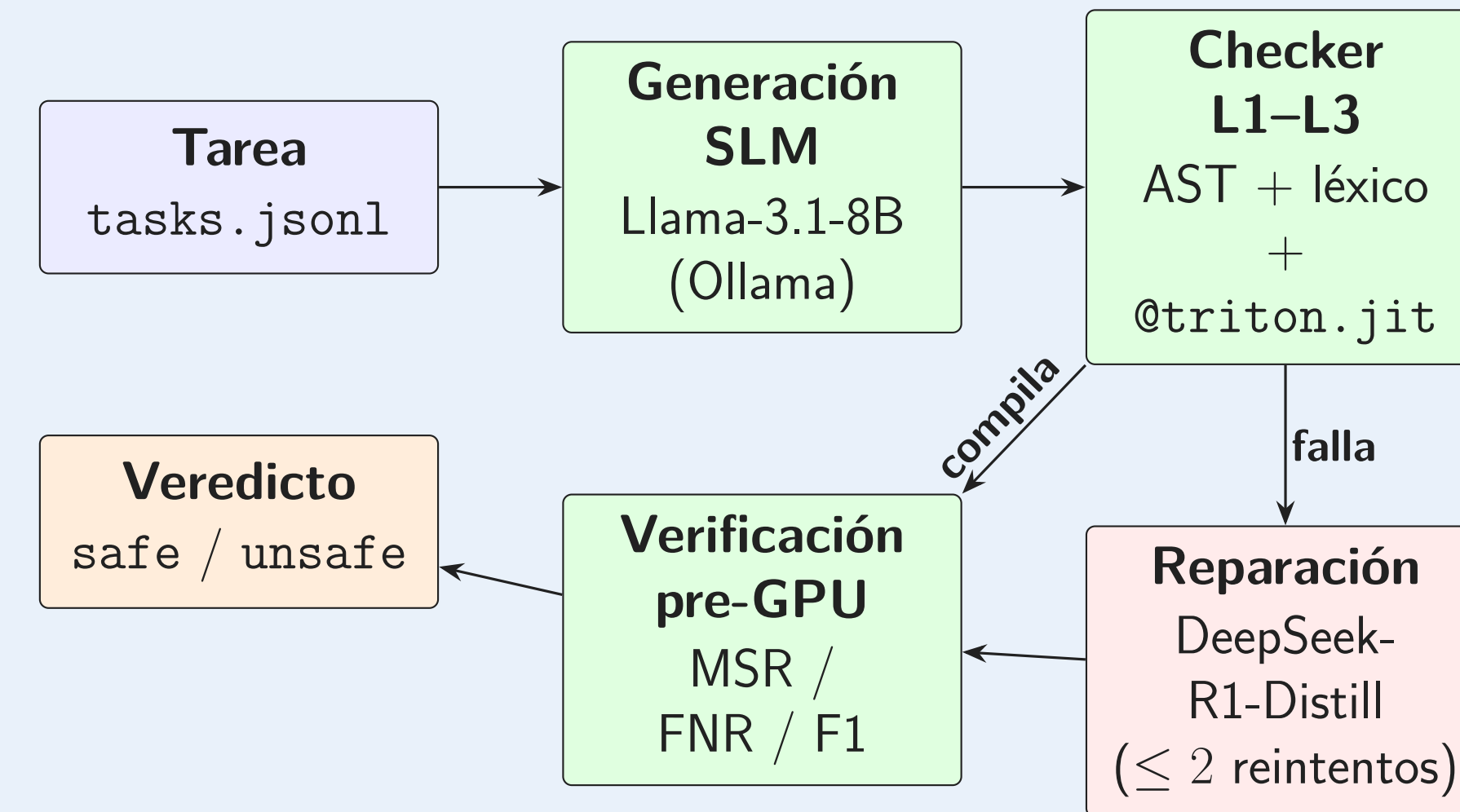


Figure 1: De una tarea a un veredicto: el SLM genera, el checker valida en tres niveles, R1-Distill repara los fallos y la verificación pre-GPU etiqueta el kernel sin compilar en CUDA.

El checker exige tres niveles en M4: **L1** sintaxis Python (AST), **L2** reglas léxicas del equipo, **L3** presencia de `@triton.jit` con `tl.load/tl.store`. Compilar aquí significa pasar L1–L3, *no* ejecutar en GPU NVIDIA.

5. Diseño experimental

Seis configuraciones comparadas con el mismo benchmark y tareas (*within-subjects*, $T = 0$, semilla 42, timeout 120 s):

- **SLM locales:** Llama-3.1-8B, DeepSeek-R1-Distill-Qwen-14B.
- **Frontera:** DeepSeek-Coder-V2, Claude 3.5 Sonnet, GPT-4o
- **Propuesto:** Triton Heal dual (Llama + reparación DeepSeek-R1-Distill-Qwen).

Contrastes pre-registrados (Holm-Bonferroni):

ID	Prueba	Comparación
C4	McNemar	Compilación: dual vs GPT-4o
C7	McNemar	Compilación: dual vs Llama
C1–C3	McNemar	Detección (MSR/FNR)
C5	Wilcoxon	Latencia frontera vs SLM
C6	Friedman	F1 entre configuraciones

6. Métricas de evaluación

- **Tasa de compilación:** % de salidas que pasan L1–L3.
 - **Speedup proxy:** $s = t_{\text{baseline}}/t_{\text{pipeline}}$, indicador operativo en M4 (no es speedup medido en GPU).
 - **MSR** (Mutant-Spotting Rate): fracción de kernels *unsafe* correctamente marcados; **FNR**: *unsafe* pasados por alto.
 - **F1 y % JSON válido:** calidad y parseabilidad de la respuesta del verificador.
- Relevancia práctica exigida: ≥ 5 pp en compilación/MSR y ~ 8 pp de reducción de FNR, además de $p_{\text{Holm}} < 0.05$.

7. Resultados

Titular: sobre 10 tareas de generación y un banco de 70 kernels (Mac M4, Ollama, semilla 42), DeepSeek-Coder-V2 fue el más confiable (**100% compilación, MSR 87.1%, FNR 12.9%**); el dual logró **MSR 80.6% / F1 0.86**, por encima de Llama solo (64.5%) en detección, sin superar al mejor frontera. Tras Holm, solo el Friedman global de F1 es significativo ($p < 0.001$).

Configuración	Comp.%	MSR%	F1	FNR%
Llama-3.1-8B (local)	50	64.5	0.81	35.5
R1-Distill-14B (local)	0	12.9	0.22	87.1
Coder-V2 (front.)	100	87.1	0.81	12.9
Claude 3.5 (heur.)	100	83.9	0.91	16.1
GPT-4o (heur.)	100	83.9	0.91	16.1
Triton Heal (dual)	50	80.6	0.86	19.4

Table 1: Descriptivos por configuración (gen. $N = 10$, verif. $N = 70$).

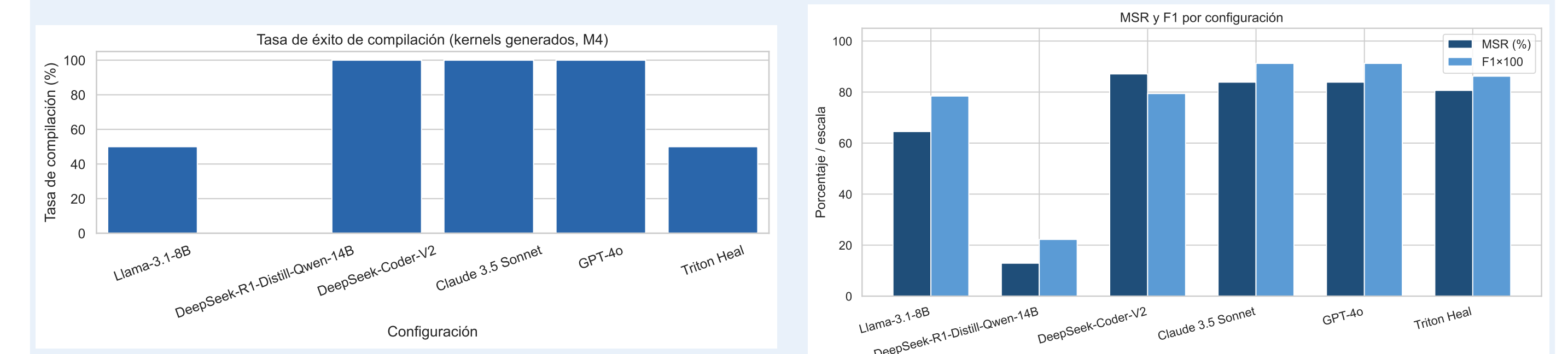


Figure 2: Izq.: tasa de compilación (%) de kernels generados. Der.: MSR (%) y $F1 \times 100$ en verificación.

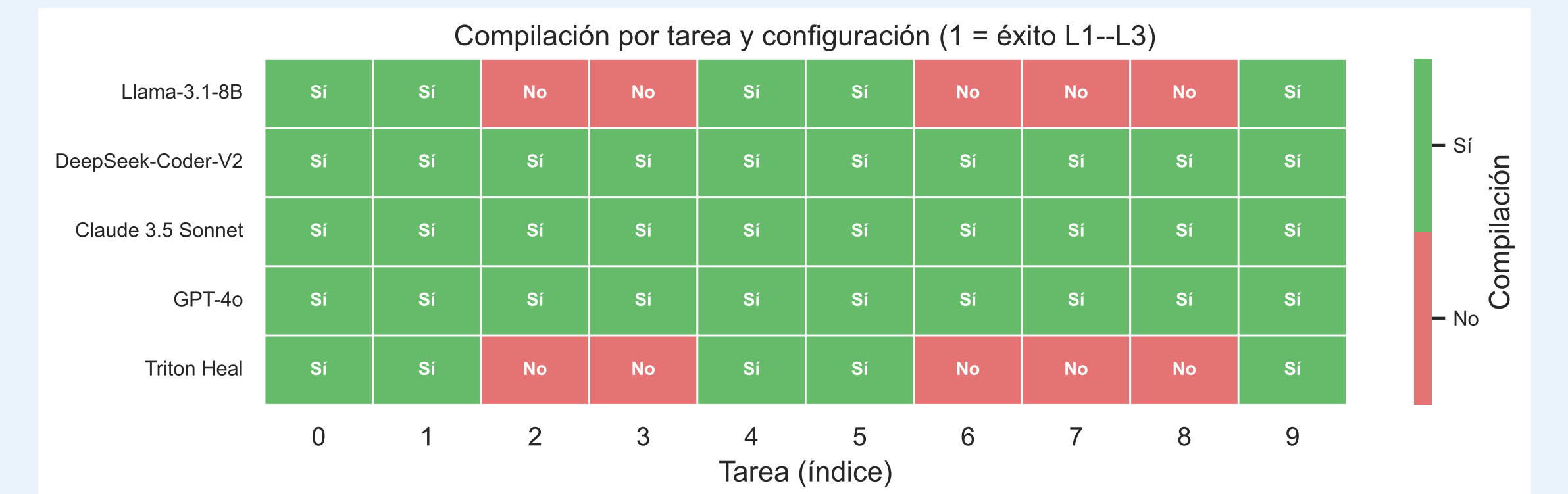


Figure 3: Mapa de compilación tarea $\times$ configuración (verde = éxito, rojo = fallo).

8. Conclusiones y trabajo futuro

- **Conclusión:** el mejor desempeño global fue de DeepSeek-Coder-V2; el dual es prometedor como *arquitectura* (reparación local + veto de frontera), no como ganador absoluto.
- **Evidencia:** tras Holm ningún McNemar rechaza H_0 (el dual no supera a GPT-4o, $p \approx 0.15$); solo el Friedman global de F1 es significativo ($p < 0.001$).
- **Limitaciones:** M4 sin CUDA (compilación *sintáctica*); muestra reducida; R1-14B inestable; baselines Claude/GPT.
- **Trabajo futuro:** APIs reales, 20 tareas $\times$ 3 réplicas, GPU NVIDIA y exportar el filtro a decodificación restringida.

Referencias

- [1] P. Tillet et al. *Triton: An IL and Compiler for Tiled Neural Network Computations*. MAPL, 2019.
- [2] Q. McNemar. *Note on the sampling error of correlated proportions*. Psychometrika, 1947.
- [3] S. Holm. *A simple sequentially rejective multiple test procedure*. Scand. J. Stat., 1979.
- [4] M. Friedman. *The use of ranks to avoid the normality assumption in ANOVA*. JASA, 1937.